

Translating Computer Languages

So far, we've focused on reading in programs and other structured text in this book. Previously, we built input models (usually ASTs) of the input and then analyzed or even executed that input by scanning the model. In this next part of the book, we're going to reverse the process and build text-to-text translators and generators that emit structured text. Translators let us implement new DSLs and programming languages quickly by translating them to existing languages. For example, the first C++ implementation (cfront) translated C++ to C. Translators are also heavily used for migrating legacy code and data formats.

A translator is a program that maps input constructs to output constructs. Sometimes we can do this purely with syntax. For example, translating a scalar multiply like $a \times b$ in a mathematics DSL to $a*b$ in Java doesn't require semantic information. If that DSL has matrices (2D arrays), though, we need to alter the translation based on type information. For example, to translate matrix multiply $A \times B$, we need to generate a nested **for** loop in Java. Things can get really tricky when we need lots of semantic information spread across an entire project. For example, doing a static field move from one class to another forces a refactoring engine to find all uses of it.

Translators vary in difficulty depending on a number of factors including the relative order of input and output constructs, the presence of forward references, whether we need to preserve comments and formatting, sheer input file size, and so on.¹ But, regardless of the translation strategy, we need to keep an important design principle in mind.

1. <http://wwwantlr.org/wiki/pages/viewpage.action?pagelid=1773>

Translation involves fully understanding each input phrase, picking an appropriate output construct, and filling it with elements from the input model. Trying to cleverly replace input symbols with output symbols rarely works well. You end up with what we call *aliteral translation* in a natural language.² For example, *faire un canard* in French means literally “to make a duck.” The real translation, though, is “to hit a wrong note.”

In a computer translator, to “understand” a phrase means to syntactically and semantically analyze it. We usually have to create an input model like an AST because we can’t always do semantic analysis properly as we parse. As we did in Chapter 8, *Enforcing Static Typing Rules*, on page 196, we have to create symbol tables and compute expression types from the input model. According to the needs of the task at hand, we compute everything we need to know about the input and then make a decision about mapping an input phrase to an output phrase.

Translation is a huge topic, and we can devote only one chapter to it, so we’re going to focus on overall strategies and a few of the most common patterns:

- Pattern 29, *Syntax-Directed Translator*, on page 307: This translator consists of a parser (or grammar) with embedded actions that immediately generate output. The key distinguishing feature is that syntax-directed translators don’t build an internal representation. They have to do everything in one pass.
- Pattern 30, *Rule-Based Translator*, on page 313: Rule-based translators use the DSL of a particular rule engine to specify a set of “this goes to that” translation rules. Combined with a grammar describing the input syntax, a rule engine can automatically perform translations.
- *Model-driven translation*: There are lots of variations within the model-driven translator strategy, but they all have one thing in common: an internal representation of the input. From the input model, a translator can emit output directly, build up strings, build up templates (documents with “holes” in them where we can stick values), or build up specialized output objects (Pattern 31,

2. I’ll never forget trying to learn French by reading comic books. It took me forever to figure out why the cartoons had “Bread! Bread!” stamped all over them during action scenes. It turns out that “*Pain! Pain!*” sounds like “Bang! Bang!” and is the way to say it in French.

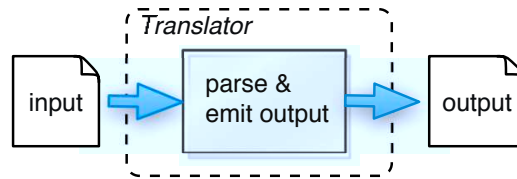


Figure 11.1: Syntax-directed translator

Target-Specific Generator Classes, on page 319). Depending on the complexity of the translation, the translator might need multiple passes over the input model before the generation phase.

Because most industrial translators use the model-driven approach, we'll spend most of our time talking about mapping input to output models. Then, as part of the patterns, we're going to implement a mock build tool, a wiki to HTML translator, and a simple SQL CREATE TABLE statement generator. Let's get the ball rolling by looking at the simplest possible translator design.

11.1 Syntax-Directed Translation

A syntax-directed translator reads input and immediately emits output as it goes. In Figure 11.1, we see the one-stage application pipeline. Even though it has few moving parts, we can use this for a surprising number of DSLs.

Syntax-directed translators consist of a grammar (or equivalent hand-built parser) and output actions. For example, here is a rule that translates a scalar multiply, such as $a \times b$, to Java:

```
mult : a=ID 'x' b=ID {System.out.println($a.text+"*"+$b.text);} ;
```

Because syntax-directed translators make only one pass through the input, they can't deal with forward references. For example, we needed multiple passes in Pattern 19, *Symbol Table for Classes*, on page 182 to handle forward references to methods and fields. They also can't perform very sophisticated translations because their application pipeline is only one stage deep. It has no opportunity to analyze an input model, which is typically required for complex translations.

Syntax-directed translators also break down when the order of input and output constructs differs significantly. For example, imagine a

translation that negates a list of integers and spits them out in reverse order. A syntax-directed translator would negate the first number and immediately spit it out. Without buffering up the negated integers, we can't deal with radically different input and output orders.

Building a syntax-directed translator means writing a lot of informal code by hand. To understand what such a translator implements, we've got to imagine the emergent behavior of all those actions. We'll explore the details in Pattern 29, *Syntax-Directed Translator*, on page 307. Let's move on to DSLs designed to make translation easier and less *ad hoc*.

11.2 Rule-Based Translation

To build a translator, we have to define an input-to-output mapping (a set of “this goes to that” rules) no matter what translator architecture we use. Instead of coding them with a general-purpose programming language, we can use a translation DSL. There are a number of excellent rule-based systems available such as ASF+SDF,³ Stratego/XT,⁴ and TXL.⁵ The ANTLR project also has a simple rule engine called ANTLRMorph.⁶

To use these engines, we feed them an input language grammar and a set of translation rules, as shown in Figure 11.2, on the following page. From the grammar, the engine builds a parse tree from the input and then applies the rules to get the output. As we saw in Section 5.4, *Decoupling Tree Traversal from Pattern Matching*, on page 125, a rule-based approach has a number of advantages. We get to focus on the input constructs (subtrees) we care about, and we don't have to specify tree-walking details.

To demonstrate the rule-based approach with a simple translation DSL, let's look at an *awk* script that extracts class names by looking for the **class** keyword:

```
/^public\ class/ {print $3} # In "public class foo", foo is 3rd element
/^class/        {print $2} # In "class foo", foo is 2nd element
```

3. <http://www.meta-environment.org>

4. <http://strategoxt.org>

5. <http://www.txl.ca>

6. <http://www.antlr.org/wiki/display/Morph/Home>

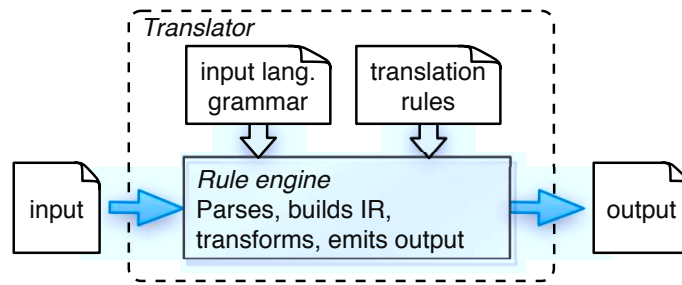


Figure 11.2: Rule-based translator

`awk` scripts are a series of patterns (regular expressions in `/.../`) and actions (in `{...}`). `awk` walks an input file line by line, executing actions associated with patterns that match the current line.

Notice that we don't have to specify an entire Java grammar to make this work. Of course, this is a simple transformation. In general, we need complete syntactic and semantic analysis like any other translation approach. Still, it's amazing what we can build with just a few pattern-action rules. For example, we're going to build a wiki-to-HTML translator in Pattern 30, *Rule-Based Translator*, on page 313.

All of these tools allow multiple translation and information-gathering passes using different sets of rules. For example, it's extremely common to have a symbol definition pass and then a symbol resolution pass. We might also need a type computation pass. All of this has to happen before we can apply the translation rules.

Rule-based systems are particularly good at legacy code conversions because we want the translated code to be natural. For example, If we see a nested loop implementation for a matrix multiply in Java, we want to recognize that pattern and convert it to a simple $A \times B$ in a mathematics DSL (not another nested loop). Rule-based systems let us just lay out all the input-output transformations we want. We can add new ones easily as we think of them.

I took advantage of this fact when writing this book. The publisher has an excellent proprietary DSL for writing books, but to reduce the strain on my hands, I designed a really terse DSL and built a translator from my language to their internal format. As I write this book, I think of new timesaving commands and add them to my list of translation rules.

This all sounds great. These rule engines let us say *what* to do, not *how* to do it. They are powerful, implementation-language independent, expressive, formal, and beautiful. It might surprise you, then, that these pure rule-based translation systems don't enjoy widespread usage in industry.

From what I can tell, developers have a number of objections. These systems tend to be complex beasts. With a large number of translation rules, translations can be slow (though we might not care). Some also find the rule engines' functional programming style too different from the imperative programming style they're used to. The rule engines themselves are black boxes, which can make it hard to understand what's gone wrong in a translation. Some of these systems were also designed to be the center of the universe, making them tricky to integrate into applications.

If you need sophisticated pattern matching that integrates well with Java, take a look at Tom⁷ (Tom also supports C, Python, C++, and C#) and the Scala⁸ programming language (which looks like Java + ML).

Next we're going to look at the translator architecture used most often in industry. It's a hybrid approach that glues together components from different formal tools. As you'd expect, as we reduce the level of abstraction, we've got more work to do. In compensation, we get to work with simpler tools and use familiar programming languages. We also get greater control over efficiency.

11.3 Model-Driven Translation

In a model-driven translator, everything centers around an input model created by the parser. In its simplest form, a model-driven translator creates an AST input model and then walks it to generate output with print statements.⁹ The only difference between this and a syntax-directed translator is that we generate text while walking a tree instead of while parsing a token stream.

The advantage of creating an AST first is that we can do semantic analysis before generating output. Most of the time, we'll need to annotate

7. <http://tom.loria.fr>

8. <http://www.scala-lang.org/>

9. The figures in this section use the more general term *intermediate representation* (IR), instead of AST.

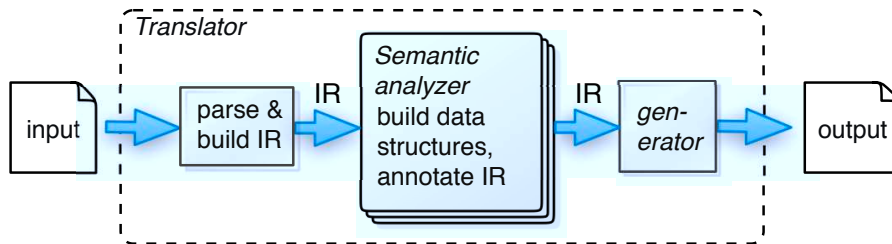


Figure 11.3: Model-driven translator

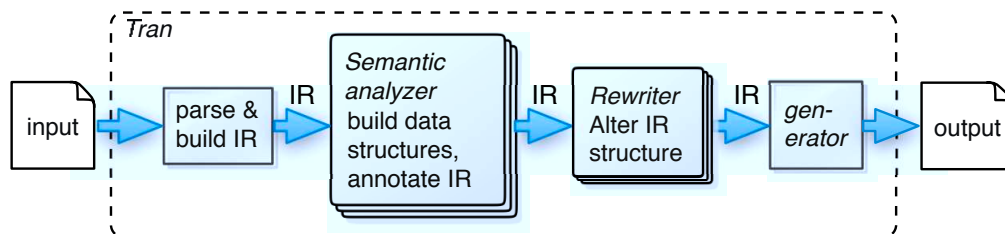


Figure 11.4: Model-driven translator with tree rewriting

the AST with symbol and type information before generating output. In Figure 11.3, we can see the overall architecture. From a software engineering point of view, it's also nice to keep output-language specific actions out of the parser. Sequestering them in a separate tree walker lets us reuse the parser for other tasks.

Once we've got an AST, we can also restructure it as part of the translation, as shown in Figure 11.4. The role of tree rewriting in translator design is a pretty big topic and too much to go into here. Unless you have a lot of experience building translators, I'd avoid rewriting the tree unless you're doing some simple optimizations like we did in Pattern 15, *Tree Pattern Matcher*, on page 138. The majority of (noncompiler) translators skip tree writing and go directly to output generation.

From the AST input model, we're going to learn how to derive an appropriate output model instead of immediately generating output. Simple print statements fall flat when the input and output phrase orders don't line up. We really need to buffer up the translated phrases and arrange them later in the proper order. As we walk the input model, we'll match subtrees and create output objects to represent translated

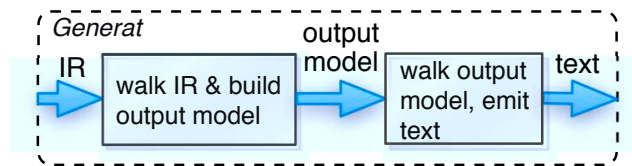


Figure 11.5: Generator for model-driven translators

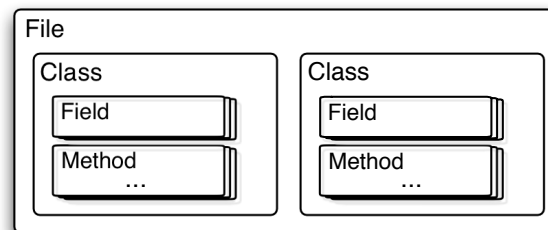


Figure 11.6: Nested output model for Java file

phrases (as summarized by Figure 11.5). These output objects can be mindless strings, templates, or custom-built objects. The key is organizing them into a proper output model.

The structure of an output model is always a nested arrangement of output objects. For example, in Figure 11.6, we see a hypothetical output model for a specific Java program. The overall File object embeds Class objects, which embed Field and Method objects, and so on. For large Java programs, this data structure can be deeply nested.

Personally, I like to think of these nested structures as trees. In Figure 11.7, on the following page, we see the equivalent tree structure, which should look familiar. It's a syntax (parse) tree. As we learned in Chapter 2, *Basic Parsing Patterns*, on page 37, a syntax tree represents the underlying syntactic structure of a phrase. My point is that using a hierarchy of output objects to represent the output makes sense because it's equivalent to a syntax tree.

In this section, we're going to learn about the different kinds of output objects and why we need to create those objects in the first place. After that, we'll dig into the mechanics of constructing output model hierarchies.

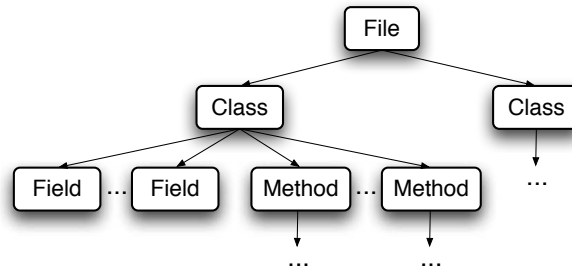


Figure 11.7: Hierarchy view of nested output model for Java file

Creating Target-Specific Generator Classes

Because we're programmers, we tend to write classes to solve problems. For example, to generate SQL CREATE TABLE statements, it seems like a good idea to design a Table class (see Pattern 31, *Target-Specific Generator Classes*, on page 319). We can then render it to text by calling its `toString()` method. This approach works and feels very comfortable.

The problem is that it's often a huge pain to create such output data structures manually. It's easier just to emit the output directly while walking the input model. For example, let's say we want to generate the Java bytecodes for a "hello world" `main()` method. To create a bytecode output model for `main()`, we can use the Byte Code Engineering Library (BCEL). Here is the sample code from the BCEL manual¹⁰ to create the method definition object:

```

MethodGen mg =
    new MethodGen(ACC_STATIC | ACC_PUBLIC, // access flags
                  Type.VOID,                // return type
                  new Type[] {              // argument types
                      new ArrayType(Type.STRING, 1) },
                  new String[] { "argv" }, // arg names
                  "main", "HelloWorld",    // method, class
                  «an InstructionList», «a ConstantPoolGen»);
  
```

That seems like a lot of work just to define a method. All of these constructor calls really add up quickly to a very large code generator. Rather than using these target-specific generator classes, it's easier to use a few print statements to generate a method definition in a byte-

10. <http://jakarta.apache.org/bcel/manual.html>

code DSL. For example, using the Jasmin¹¹ bytecode assembler, here's what `main()` looks like:

```
.method public static main([Ljava/lang/String;)V
    ...
.end method
```

This is much more clear. Of course, we have to learn a tiny bit of syntax, but that's easier than learning the vagaries of a library such as BCEL. The only strange bits are the `[Ljava/lang/String;` and `V` strings, which are Java type encodings¹² representing an array of `String` objects and `void`, respectively.

Aside from being a hassle, it's usually not worth the development and run-time cost to create a data structure when the input model itself is close enough to be workable. If we have an AST subtree, for example, that represents the `main()` method, there's no point in writing a class like `MethodGen`. Here's a sample AST (Pattern 11, *Irregular Heterogeneous AST*, on page 114) node for a method definition:

```
class MethodNode extends StatNode {
    String name;
    Type returnType;
    List<String> modifiers; // public, static, final, ...
    List<ArgNode> args;
    BlockNode body;
    ...
}
```

From that, we can directly generate the Jasmin code, letting Jasmin do all of the work to emit the `.class` file:

```
void gen(MethodNode m) { // method node visitor
    System.out.print(".method ");
    «print m.modifiers»
    System.out.print(m.name+"(");
    for (ArgNode a : m.args) gen(a); // visit the arguments
    System.out.print(")");
    «println m.returnType»
    gen(m.body); // visit the method body
    System.out.println(".end method");
}
```

This principle also applies when we are generating high-level source code, not bytecodes. For example, rather than building up objects that

11. <http://jasmin.sourceforge.net>

12. http://java.sun.com/docs/books/jvms/second_edition/html/ClassFile.doc.html#1169

represent Java program phrases such as `WhileStatement` and `AssignStatement`, we can just spit out Java code by walking the AST. Here is how we could generate code for Java assignments using Pattern 13, *External Tree Visitor*, on page 131:

```
class AssignNode extends ExprNode { String id; ExprNode valueExpr; }
...
void gen(AssignNode n) {
    System.out.println(n.id+"="); // emit left-hand-side and '='
    gen(n.valueExpr);             // walk right-hand-side expression
    System.out.println(";");      // emit final ';'
}
```

OK, so we can save some work by generating text on the fly while walking a suitable input model instead of creating an output model. The bad news is that print statements don't always work.

Print Statements Are Easy but Inflexible

Visitors that directly emit text with print statements work fine as long as the order of input and output constructs is very similar. The order in which we visit input model nodes has to match the order in which we need to generate output.

This turns out to be pretty restrictive. Take Java to C translation, for example. Because C doesn't allow forward references, we need to generate a list of function declarations at the top of the output C file:

```
extern int f(float y);      // f declaration
extern void g();           // g declaration
int f(float y) { g(); }    // f definition; forward reference to g()
void g() { ... }          // g definition
```

Without the `extern` declaration of `g()`, a C compiler would complain about the call to `g()` inside `f()`. So, for each function, we need to emit both a declaration and a definition. Our first attempt might look like this:

```
void gen(MethodNode m) {
    «compute and emit extern C declaration»
    «translate Java body and emit C function»
}
```

Unfortunately, this won't work. The output would interleave the declarations and definitions:

```
extern int f(float y);      // f declaration
int f(float y) { g(); }    // f definition; forward reference to g()
extern void g();           // g declaration; TOO LATE!
void g() { ... }          // g definition
```

Oops. We need all the declarations first. Somehow, we have to separate when we translate phrases from when we spit them out.

Decoupling Input Model Traversal from Output Order

There are two ways to solve input-output ordering mismatches. First, we can walk the tree twice, once for declarations and again for definitions using two visitor methods:

```
void genDecl(MethodNode m) {«compute and emit extern C declaration»}
void genDef(MethodNode m) {«translate Java body and emit C function»}
```

This works but is inefficient because we have to walk the (potentially very large) tree twice. This is an *output-driven* approach because it “pulls” information from the input model according to the output order.

The second choice is to walk the input model a single time, collecting the declarations and definitions in lists instead of directly emitting text:

```
void gen(MethodNode m) {
    «compute and add extern C declaration to a list»
    «translate Java body and add C function to a list»
}
```

Once we’ve got those two lists, we can emit the proper C file by printing the lists one after the other. This *input-driven* approach lets us decouple the input and output order, albeit at the cost of buffering up the various output pieces. It’s totally worth it, though. One of the most important lessons I’ve learned over the years is that *we should compute information and translate phrases when it’s convenient and efficient to do so, not when the output order demands it*.

Let’s see what the input-driven approach looks like. Instead of printing on the fly, we’ll create strings. Here’s a visitor method that returns a string representing an assignment:

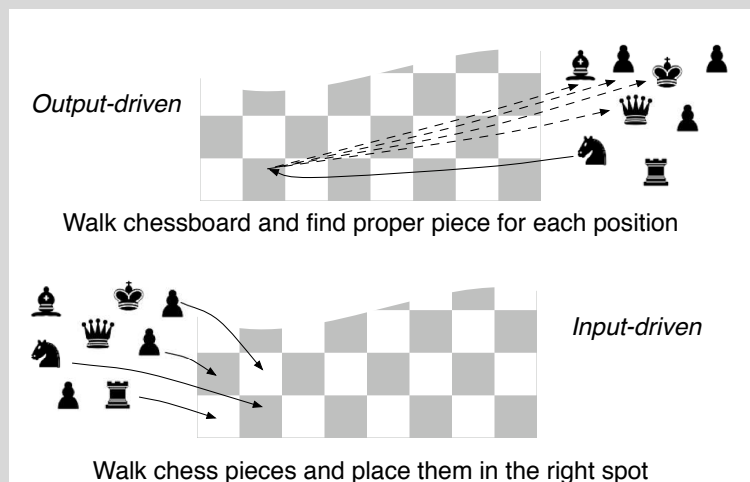
```
String gen(AssignNode n) {
    String e = gen(n.valueExpr); // walk right-hand-side expression
    return n.id + "=" + e + ";";
}
```

Or, instead of writing the AST visitor manually, we could use Pattern 14, *Tree Grammar*, on page 134:

```
assign returns [String s]
    :   ^('=' ID e=expression) {$s = $ID.text + "=" + $e.s + ";"}
    ;
expression returns [String s] : ... ;
```

Chess Analogy for Input- vs. Output-Driven Generation

Setting up a chessboard involves putting the chess pieces in specific locations on the board. We can think of the jumble of chess pieces as the input model (an unsorted list of pieces). The output is a 2x8 matrix, assuming we're setting up only one player's side. The output-driven approach marches through the output matrix position by position. At each spot, it scans through the input model looking for the right piece. In contrast, the input-driven approach simply places pieces in the right spot as it encounters them:



The dashed lines signify the output-driven generator searching for the right piece.

The point is that we've created the same text as we did in the previous section. We just aren't emitting it as we compute it. Whoever calls method `gen()` or rule **assign** can decide what to do with the translated text.

Let's review what we've learned so far. We know that the translation process is about creating an input model, enriching it with semantic information, and then creating an appropriate output model. Target-specific generator classes are familiar and well-structured, but building them is a lot of work, and they're often a hassle to use. Visitors that generate text directly are much more convenient. Unfortunately, print statements lock the order of the output elements to the input model

traversal order. Besides, computing output strings in a general-purpose programming language ain't exactly pretty.

All is not lost, though. We can use templates, which are the best of both worlds. A template is a text document with “holes” that we can fill in. Templates are specialized generator objects that support a DSL designed for generating structured text. We'll explore templates in great detail in the next chapter.

In the meantime, let's finish off our discussion of the translation process. We need to learn some of the engineering details involved in creating output models.

11.4 Constructing a Nested Output Model

To build a translator, we need to look at the mechanics of matching an input construct and creating the right output object. Then, we have to figure out how to organize all those translated pieces into a nested hierarchy.

Creating Output Objects from Input Phrase Components

Translating an individual input phrase means creating the appropriate output model object and injecting it with elements from the input phrase. Here's what the assignment node example from the previous section looks like using an AST visitor and a target-specific generator object:

```
Statement gen(AssignNode n) {
    Expression e = gen(n.valueExpr); // walk right-hand-side expression
    return new AssignmentStatement(n.id, e);
}
```

Notice that we're not specifying the actual output literals. That would hard-code the translation. It would also make our translator hard to read because we'd be mixing the syntax of two languages (that of the implementation language and that of the output language). When using such explicit generator classes, the “pipe” between input and output model is usually the constructor arguments.

Mapping an assignment subtree to an assignment output object is reasonably clear, so let's try something a little harder: translating scalar and matrix multiplication. The syntax is the same for both, but semantically, they are very different. So, our translation method has to test the type of these subexpressions in order to create the proper output object.

```
// Match scalar a x b and matrix A x B multiplies for a math DSL
// Assume nodes contain result types. match ^('x' expression expression)
Expression gen(MultiplyNode n) {
    Expression a = gen(n.left);
    Expression b = gen(n.right);
    if ( «subexpressions have matrix types» )
        return new MatrixMultiply(a, b);
    else
        return new ScalarMultiply(a, b);
}
```

The «*subexpressions have matrix types*» conditional would test the subexpression types stored in the AST nodes like we did in Pattern 22, *Enforcing Static Type Safety*, on page 216.

To move things around locally within a phrase, all we have to do is change some parameters. For example, we can change the order of scalar multiplication operands by swapping the order of constructor parameters: `ScalarMultiply(b, a)`. To send a translated phrase to a radically different spot in the output, though, we need to figure out how to organize the overall output model.

Organizing Translated Phrases into a Nested Model

To create an output model, we have to build up a nested data structure composed of output objects. (For the purposes of this discussion, we'll continue using specialized generator objects.) Let's start with the easiest case where the input and output element orders match up.

Let's say we've got a Java method AST input model and want to create a C# output model. Using a visitor, we could have all the methods return an output object. An invoking visitor method would incorporate the return value into its own output object. For example, given a `MethodNode`, the following visitor method returns a `Method` object representing a C# method.

```
Method gen(MethodNode m) { return Method(m.id, gen(m.bodyBlock)); }
```

This visitor method injects the name and translated body returned from `gen(m.bodyBlock)` into the `Method` instance. The following visitor method handles the body translation:

```
Block gen(BlockNode b) {
    List<Statement> stats = new ArrayList<Statement>();
    for (StatementNode s : b.stats) stats.add( gen(s) );
    return new Block(stats);
}
```

Whetting Your Appetite for Templates

As a teaser for the next chapter, here's how we'd use a template and an ANTLR tree grammar to map input patterns to output constructs:

```
assign:
  ^('=' ID e=expression) -> assign(id={$ID.text}, e={$e.st}) ;
```

The rule matches an input assignment tree and creates an instance of the template called **assign**. The **assign** template definition might look like this:

```
assign(id, e) ::= "<id> = <e>";
```

The names inside angle brackets are the "holes" we can fill in. Templates are like generator classes except that we only use a single class called `StringTemplate`.

For mapping $a \times b$ to the appropriate output constructs, we could use the following tree grammar rule with template constructors and a predicate:

```
// Match scalar a x b and matrix A x B multiplies in a math DSL
// Assume each node knows its result type. Each
// rule also implicitly returns a template output model object.
mult:  ^('x' a=expression b=expression)
      -> {«subexpressions have matrix types»?
          matrixMult(a={$a.st}, b={$b.st})
      -> scalarMult(a={$a.st}, b={$b.st})
      ;
```

It queues up the translated statements and injects them into the `Block` return value. Among the statement subtree visitor methods, we'd have something like the following to translate assignments:

```
AssignStatement gen(AssignNode a) {
    return new AssignStatement(a.id, gen(a.expr));
}
```

Now let's deal with the situation where the output element order differs from the input order. As we saw in Section 11.3, *Decoupling Input Model Traversal from Output Order*, on page 301, translating Java to C code requires that we generate a declaration for each function definition. Most important, those declarations must appear before all the function definitions. To translate a Java class to C, we need to generate the following strict sequence.

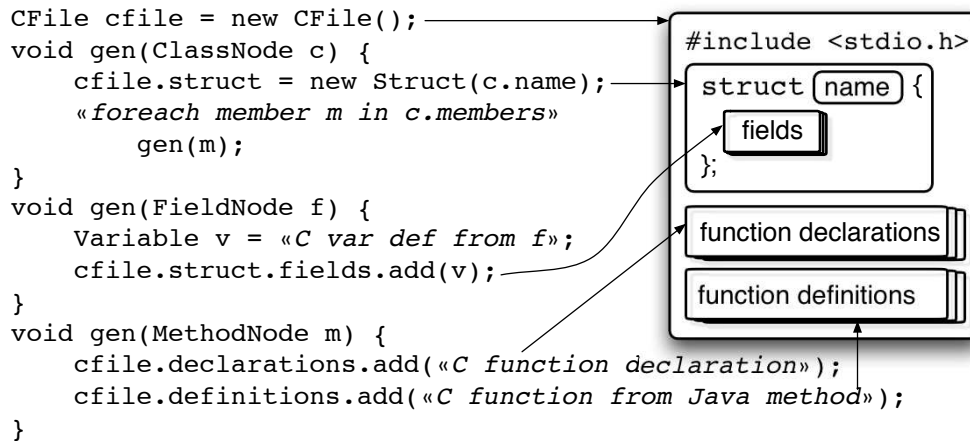


Figure 11.8: Assembling a C file model using Java AST visitor

```

#include <stdio.h>
«struct»          // has all fields extracted from Java class
«declarations»    // function declarations from Java methods
«definitions»     // function definitions from Java methods

```

The output model is shown graphically in Figure 11.8, which also shows how visitor methods would fill the model. One of the key elements of the code fragment is the “global” `cfile` variable. In general, translators need to track a few locations in the output model. These locations are typically things such as the current file object, the current class object, or the current method object. It’s like remembering where the refrigerator and spice cabinets are in your kitchen. They tell you where to put things and effectively organize your kitchen.

The visitor method for `ClassNodes` indirectly triggers the other two visitor methods for field and method nodes. Those methods need to inject data into the model starting at the root (the outermost output object). The visitor methods don’t bother returning output objects because they can directly inject elements. (In my experience, you’ll have some visitor methods that return output objects and some that don’t.)

When the visitor finishes walking the input model, we can generate code by calling `toString()` on the root, `cfile`. That, in turn, renders its embedded objects with `toString()`, and so on, recursing down the output object hierarchy. And with that `toString()` call, we finish the story of how we translate text using the model-driven approach.

At this point, you're familiar with the three translation strategies. You've also learned a lot about the model-driven approach. We learned that print statements aren't always powerful enough but that they are much easier to use than writing specific output classes. In the next chapter, we'll see that templates are often a good compromise between the two. They let us construct the nested output models we need without having to define a new class for every output construct. Before we get there, though, let's formally define the patterns we've talk about in this chapter:

Pattern

Pattern 29, *Syntax-Directed Translator*

When to Apply

This pattern is the simplest mechanism because it uses print statements to generate output rather than creating an output model. Use this pattern if you can get away with it. In other words, if you don't need information appearing later in the input, syntax-directed translation works. These translators also cannot generate output in a radically different order from the input.

Pattern 30, *Rule-Based Translator*, on page 313

Rule-based systems are elegant but often difficult to learn. They are also black boxes, which can make it difficult to debug translators. At the very least, it makes some programmers uncomfortable. In the hands of programmers with very strong theoretical backgrounds, these systems often prove extremely powerful.

Pattern 31, *Target-Specific Generator Classes*, on page 319

This pattern represents the final output model for a model-driven translator. Each output language construct corresponds to a specific class. The fields of an output object track any embedded output objects.



Purpose

This pattern generates text using a grammar, or equivalent hand-built parser, embedded with actions.

Discussion

Syntax-directed translators are little more than grammars salted with actions (code snippets). They don't build internal models and then walk them to generate output.

Putting actions directly in a grammar sometimes makes the grammar difficult to read. It also locks the grammar into a single bit of translator functionality. It's often a good idea to disentangle actions from the grammar by following what we did in Pattern 26, *Bytecode Assembler*, on page 265 (another example of a syntax-directed translator). The assembler grammar defined placeholder methods that describe the translator functionality:

```
protected void gen(Token instrToken) {}
protected void gen(Token instrToken, Token operandToken) {}
protected void defineFunction(Token idToken, int nargs, int nlocals) {}
...
```

Then, the actions in the grammar invoked these methods to trigger translation:

```
instr
:   ID NEWLINE           {gen($ID);}
|   ID operand NEWLINE  {gen($ID,$operand.start);}
...
;
```

To implement the translation, we created a subclass that defined the concrete methods:

```
/** Subclass the AssemblerParser to actually implement the necessary
 * symbol table management and code generation functions. */
public class BytecodeAssembler extends AssemblerParser {...}
```

If we wanted to make it easier to change the behavior of the translator on the fly, we could pass in a strategy object. We'd move the placeholder methods into an interface such as *AssemblerBehavior*. The actions would then look like this: `behavior.gen($ID)` instead of just `gen($ID)`. We'd pass `behavior` in from the object using the assembler. In the next section, we'll follow this approach for our sample implementation.

Implementation

Let's now implement a make-like build dependency DSL that translates input like the following to an equivalent Java program:

```
«target-file» : «dependencies»
    «action(s) that create target-file»
```

For example, here are some rules from the sample makefile included in the source directory:

```
t.o : t.c
    gcc -c -o t.o t.c
go: t.o u.o
    gcc -o go t.o u.o
    echo done
```

Our goal is to generate `makefile.java` from file `makefile`. The entire process looks like this:

```
$ javac *.java          # build translator tool
$ java Maker makefile   # translate makefile to makefile.java
$ javac makefile.java
$
```

Once we've got an executable version of `makefile`, we can ask it to build the targets. By asking it to make target `go`, it has to first build the `t.o` and `u.o` dependencies. Then it can create the `go` file:

```
$ java makefile go      # build target "go"
build(t.o): gcc -c -o t.o t.c
build(u.o): gcc -c -o u.o u.c
build(go): gcc -o go t.o u.o
build(go): echo done
done
$
```

If we run the same target again, it gives no output because `go` now exists and is newer than its dependencies:

```
$ java makefile go      # nothing to do now
$ java makefile clean   # wipe out all files
build(clean): rm t.o u.o go
$ java makefile clean   # oops, files are already cleaned up
build(clean): rm t.o u.o go
rm: t.o: No such file or directory
rm: u.o: No such file or directory
rm: go: No such file or directory
$ java makefile go      # build again
build(t.o): gcc -c -o t.o t.c
...
$
```

As you can see, `makefile` collects and emits any standard output and standard error from the actions (processes) it executes.

Building Support Code and Defining the Translation

There are two key components to implementing this DSL: the translator itself and the run-time support. Before we can figure out how to build

the translator component, we need to create the run-time support and manually translate a make file to Java code.

Let's create an object to represent a make target, with all of its dependencies and actions:

[Download](#) `trans/make/Target.java`

```
public class Target {
    String name;
    List<String> actions = new ArrayList<String>();
    List<String> dependencies = new ArrayList<String>();
}
```

Then we need a dictionary that maps target names to Target objects and a build() method to build a particular target. Let's stick all of this in MakeSupport. The details of walking the dependency list and checking file system timestamps aren't important here. We can assume the support code just works.

Now, let's figure out the translation. We can get away with a syntax-directed translator because we can generate appropriate Java code as we read the build instructions. Let's start by mapping a sample target to Java code. Given the `t.o` target rule, we need to generate something like this:

```
target = new Target("t.o");
target.addDependency("t.c");
target.addAction("gcc -c -o t.o t.c");
targets.put("t.o", target);
```

We also need to wrap that in a method within a class like this:

```
import java.io.IOException;
class «makefile-name» extends MakeSupport {
    public «makefile-name»() throws IOException {
        Target target = null;
        «code-for-targets»
    }
}
```

The name of the class is the same as the input file, such as `makefile`. Also in that class, we need to make a `main()` that creates an instance of the generated object. Then, we can ask it to build a target specified on the command line:

```
makefile m = new makefile(); // create Target dictionary
int r = m.build(args[0]);    // build the target
System.exit(r);              // exit with code from target build
```

So, that's what the translation looks like. Now we have to build a grammar that automates it by triggering code generation actions.

Building the Translator

With some foresight, we can spec out the list of actions to trigger and put them into an interface:

[Download](#) `trans/make/MakeGenerator.java`

```
public interface MakeGenerator {
    public void start();
    public void finish();
    public void target(String t);
    public void dependency(String d);
    public void action(String a);
    public void endTarget(String t);
}
```

This means we have to pass a MakeGenerator object into the ANTLR-generated parser:

[Download](#) `trans/make/Make.g`

```
@members {
MakeGenerator gen;
public MakeParser(TokenStream input, MakeGenerator gen) {
    super(input);
    this.gen = gen;
}
}
```

Let's write a grammar for our make-like DSL. A make file looks like a series of rules:

[Download](#) `trans/make/Make.g`

```
rules
:   {gen.start();} rule+ {gen.finish();}
;
```

To generate the header and trailer of the Java class, we trigger start() and finish() methods.

Each rule has a target file name, an optional list of file dependencies, and at least one action:

[Download](#) `trans/make/Make.g`

```
rule
:   target=ITEM ':'      {gen.target($target.text);}
    (i=ITEM {gen.dependency($i.text);})* '\n'
    (ACTION {gen.action($ACTION.text);}+
     {gen.endTarget($target.text);}
    | '\n' // ignore blank lines
    ;
```

Once we match the target name, we trigger `target()`. Then, we trigger `dependency()` for each file dependency and `action()` for each action we find. The `endTarget()` trigger gives us the hook to generate the `put()` to the targets dictionary.

The code generation implementations are messy (without templates) but straightforward. For example, here is what we generate upon the `start()` trigger:

[Download](#) `trans/make/JavaGenerator.java`

```
public void start() {
    out.println(
        "import java.io.IOException;\n" +
        "class "+makefile+" extends MakeSupport {\n" +
        "    public "+makefile+"() throws IOException {\n" +
        "        Target target = null;\n"});
}
```

The majority of the methods are much smaller. Here's how we respond to `target()`:

[Download](#) `trans/make/JavaGenerator.java`

```
public void target(String t) {
    t = t.trim();
    out.println("\ttarget = new Target(\""+t+"\");");
}
```

The translator `main()` program lives in `Maker`. It creates the ANTLR-generated parser and lexer as usual but passes in a `JavaGenerator`:

[Download](#) `trans/make/Maker.java`

```
JavaGenerator gen = new JavaGenerator(makefileName);
MakeParser p = new MakeParser(tokens, gen);
try { p.rules(); } // parse, triggering code generation actions
```

Other than support code, that's all there is to it. Our parser directly generates Java code that implements a make DSL. We compile it and run it like any other Java program.

Related Patterns

This pattern is sort of a degenerate version of a model-driven translator. We can think of the tokens as the input model and the generated text as the output model.

Building an Interpreted Version

Some of you might be wondering how we could interpret rather than translate a makefile. The idea would be to create a dictionary full of Target objects as an internal model. We'd need grammar actions to create Target objects and set dependencies and actions:

```
rule:  target=ITEM ':'
      {Target t = new Target($target.text);}
      (i=ITEM {t.addDependency($i.text);}) * '\n'
      (ACTION {t.addAction($ACTION.text);}) +
      {targets.put(t.name, t);}
      | '\n' // ignore blank lines
      ;
```

30 Rule-Based Translator



Purpose

A rule-based translator expresses a translation with a set of “x becomes y” rules, written in the DSL of a pattern-matching engine.

Discussion

To use a rule-based system, we have to feed it two things: a grammar that describes input sentences and a set of translation rules. The reason we need the grammar is to get a Pattern 8, *Parse Tree*, on page 105. The parse tree concretely defines what the various phrases and sub-phrases are. That's how they can avoid applying transformations inside comments in the input stream, for example. Although it looks like we're doing text-to-text transformations, the underlying engine is actually doing tree rewrites.

For complicated translators, we need to build a lot of ancillary data structures beyond trees such as symbol tables and control-flow graphs (that tell us which statements can get to other statements). We can do all of that and then have the engine execute our transformation rules based upon these data structures.

In practice, I use rule-based translators when I don't have or don't need a full grammar. Most of the time I'm extracting or translating just some

Syntax	Description
«first-line-of-file»	The first line of text in the file is the title of the page.
«text»	Bold «text».
«text»	Italics «text».
@«url»@	Link to «url».
@«url» «text»@	Link to «url» and make the link text be «text».
### «title»	<h1> header.
## «title»	<h2> header.
# «title»	<h3> header.
* «item»	A bullet list item. The * must start at the left edge.
[«rows»]	A table full of rows, separated by -- on a line by itself. Separate columns with .

Figure 11.9: A simple wiki syntax

of the input phrases. To work without a full grammar describing the entire input, the elements have to be easy to identify lexically (with special characters) or with a simple grammatical structure. For example, the ANTLR website has an example fuzzy Java parser that extracts class, method, variable definitions, and method call sites.¹³

Since we've already looked at tree transformations, let's see what we can do with a set of lexical rules.

Implementation

Let's translate the syntax of a simple wiki to HTML. This is actually sort of tricky, which you might not expect. Translating `*foo*` to `<b>foo</b>` doesn't seem that hard. But, bullet lists and tables take some thought.

Defining a Wiki Syntax

Let's start by defining some wiki syntax in Figure 11.9; then we'll look at a sample translation. File `t.wiki` in the source directory exercises the various wiki features. With a little help from a CSS file, our translator generates HTML that renders like the image in Figure 11.10, on the next page.

13. <http://www.antlr.org/download/examples-v3.tar.gz>

Agenda

Ad eundum quo nemo ante iit. Age. Fac ut gaudeam. Altissima **quaeque** flumina minimo sono labi

Ipsissima verba Ipsissima verba Ipsissima verba Ipsissima verba Ipsissima verba Ipsissima verba Ipsissima verba Ipsissima verba Ipsissima verba

Alibi

Inventas vitam iuvat excoluisse per artes. Inventas vitam iuvat excoluisse per artes. Inventas vitam iuvat excoluisse per artes translator with a series of lexical rules. Link to [antlr site](#)

Alias

- E pluribus unum
- Emeritus
- Ergo

Vinum bellum iucunumque est, sed animo corporeque caret. Vinum bellum iucunumque est, sed animo corporeque caret. Vinum bellum iucunumque est, sed animo corporeque caret

Ipsa facta	Ipsa facta	Ipsa facta. Ipsa facta. Ipsa facta.	
Ipsa facta. Ipsa facta	<i>newlines are ok</i>	Verbatim Verbatim Verbatim	
Veni, vidi, vici	Verso Versus	nested table	Ipsa facta. Ipsa facta
		Ventis secundis, tene cursum	Vide ut supra
foo	bar	blort	

Vinum bellum iucunumque est, sed animo corporeque caret.

Figure 11.10: HTML translated from t.wiki

Our translator treats the first line of text as the title of the page. Within the file, we need to put a blank line before paragraphs, bullet lists, sections, and tables. Here's a sample bullet list:

```
* E pluribus unum
* Emeritus
* Ergo
```

A blank line terminates the list. Tables live inside square brackets. The -- row separator has to be on a line by itself to make it more obvious. | is the column separator. Here's a sample table:

```
[
row 1 col 1 | row 1 col 2
--
row 2 col 1 | row 2 col 2
]
```

You can take a look at the t.wiki file to see a nested table and examples of italics elements, and so on. Any text other than the wiki syntax should pass through unchanged. That's one of the benefits of this method. We don't have to specify the syntax of the entire file, just the parts we care about.

This isn't the most sophisticated wiki in the world, but it's complicated enough to give the rule-based approach a workout. This example is

highly ANTLR-specific, but any rule-based solution will have to deal with the DSL and details of a particular tool.

Defining the Translation Rules and Main Program

Our implementation consists of only two files: a grammar full of lexical rules and a main program that sets things up. Here's the core of the main program:

[Download](#) `trans/wiki/WikiToHTML.java`

```
header(out);
Wiki lex = new Wiki(new ANTLRReaderStream(fr), out);
try {
    Token t = lex.nextToken();
    while ( t.getType() != Token.EOF ) t=lex.nextToken();
}
finally { fr.close(); }
trailer(out);
```

Since this is still a lexer, we need to loop around a call to `nextToken()`. This forces the lexer to consume all the input. Before and after this loop, we call `header()` and `trailer()` to spit out the HTML file header and trailer. The grammar will be an ANTLR lexer grammar with option `filter` turned on. This tells ANTLR to try the rules in order. So, if an input chunk matches more than one rule, it will match to the first rule. As a catchall, we've got an **ELSE** rule at the end that copies the input character to the output:

[Download](#) `trans/wiki/Wiki.g`

```
ELSE:    c=. {out.print((char)$c);} ; // match any char and emit
```

There's a number of interesting things going on in this grammar:

- We're using semantic predicates to turn rules on and off depending on the line number or character position within a line. Syntax `{«expr»}? is a semantic predicate that provides extra information about the viability of a rule. Since ANTLR uses semantic predicates only if syntax alone is insufficient, we need to use a special version of a semantic predicate: {«expr»?=>. The extra => forces ANTLR to always test the predicate when choosing viable rules.`
- We need to keep some state information like “we're in a bullet list.”
- We use recursion (in a lexer) to allow nested tables. Rule **TABLE** calls **ROW**, which calls **COL**, which matches a bunch of **TABLE_CONTENT**. Rule **TABLE_CONTENT** matches everything up until the end of a column. Within that text, it looks for nested tables and bold or italics items.

Our first rule matches the title by matching a single line of text as long as it occurs on the first line of the file:

[Download](#) `trans/wiki/Wiki.g`

```
TITLE
:   {getLine()==1}?=> TAIL
    {out.println("<title>"+$TAIL.text+"</title>");}
;
```

Rule **TAIL** is a helper (**fragment**) rule that matches any character other than a newline:

[Download](#) `trans/wiki/Wiki.g`

```
fragment
TAIL : ~'\n'+ ;
```

ANTLR tries only the nonfragment rules when looking for a match. Normal (nonfragment) rules have to call fragment rules explicitly. **TAIL** is in a separate rule because it's convenient to say `$TAIL.text` in **TITLE**.

Since both the bold and bullet list items start with `*`, we have to distinguish them. According to our syntax, bullet list items have to start at the left edge. So, rule **BOLD** is only applicable when it's not on the left edge:

[Download](#) `trans/wiki/Wiki.g`

```
BOLD:   {getCharPositionInLine()>0}?=>
        '*'                               {out.print("<b>");}
        (c=~'*' {out.print((char)$c);}+
        '*'                               {out.print("</b>");}
        ;
```

This rule matches `*` followed by any non-`*` characters followed by a terminating `*` character. As it matches characters, it spits them to the output stream.

Many of our wiki constructs have to appear after a blank line (bullet lists, sections, tables, and regular paragraphs):

[Download](#) `trans/wiki/Wiki.g`

```
BLANK_LINE
:   '\n\n'                               {out.println("\n"); closeList(); }
    (
      UL
    |  SECTION
    |  TABLE
    |  /* paragraph */ {out.println("<p>");}
    )
;
```

The call to `closeList()` makes sure that any prior lists have their terminating `</ul>` tag. We can easily generate the starting `<ul>` tag when we match the first bullet list item after a blank line. Rule **BLANK_LINE** invokes **UL** after matching `\n\n`. Here's **UL**:

[Download](#) [trans/wiki/Wiki.g](#)

fragment

```
UL: '*' '{out.print("<ul>\n<li>"); context.push("ul");} ;
```

The rules track context using a context stack. We only need to worry about bullet list context, but we could add numbered lists and so on. When the stack is empty, we're not in a special context. A blank line after a bullet list item terminates the list. At that blank line, then, we need to check for unterminated lists, which is the reason for the call to `closeList()` in **BLANK_LINE**. That method pops the top context and ends that element by emitting the proper HTML tag:

[Download](#) [trans/wiki/Wiki.g](#)

```
Stack<String> context = new Stack<String>();
void closeList() {
    if ( context.size()==0 ) return;
    String list = context.pop();
    out.println("</"+list+">");
}
```

The bullet items beyond the first one can emit `<li>` tags without worrying about context:

[Download](#) [trans/wiki/Wiki.g](#)

```
LI: {getCharPositionInLine()==0}?=>'*' '{out.print("<li>");} ;
```

Now let's figure out how nested tables work. We need recursive rules, which ANTLR fortunately supports even among lexical rules. The overall table structure looks like this:

[Download](#) [trans/wiki/Wiki.g](#)

fragment

```
TABLE
: '['                                     {out.print("<table border=1>\n");}
  ROW ('\n--\n' ROW)* '\n'
  ']'                                     {out.print("\n</table>");}
;
```

Eventually, **TABLE** invokes **TABLE_CONTENT**, which can include another (nested) table.

[Download](#) trans/wiki/Wiki.g

fragment

```
TABLE_CONTENT
:   TABLE
|   BOLD
|   ITALICS
|   {!upcomingEndOfCol()}?=> c=. {out.print((char)$c);}
;
```

The last alternative matches anything not matched by the other alternatives. The predicate `!upcomingEndOfCol()` ensures that we match characters only until we see the end of a column or table coming. `upcomingEndOfCol()` manually checks up to three characters ahead to make sure we don't consume too much in **TABLE_CONTENT**:

[Download](#) trans/wiki/Wiki.g

```
boolean upcomingEndOfCol() {
    return input.LA(1)=='/' ||
           (input.LA(1)=='\n' &&
            (input.LA(2)=='-' && input.LA(3)=='-') || input.LA(2)==' ');
}
```

This sample implementation illustrates what a rule-based system looks like, but it's nowhere near as sophisticated as the rule engines we mentioned in the introductory material for this chapter. That said, this lexical filter mechanism is easier to learn and still very useful.

Related Patterns

This pattern lets us specify only those patterns we care about, just like we did with trees in Pattern 15, *Tree Pattern Matcher*, on page 138. Most rule-based translation engines internally use Pattern 8, *Parse Tree*, on page 105. The sample implementation for this pattern uses an *LL(k)* version of Pattern 2, *LL(1) Recursive-Descent Lexer*, on page 49.

31

Target-Specific Generator Classes



Purpose

This pattern describes a class library whose sole purpose is to represent and generate output constructs in a particular language.

Discussion

Rather than use print statements to generate programs or data, generator classes let us insulate application code from the exact syntax of an output language. In that sense, they are similar to templates. The difference is, we're defining a special class for each output element, whereas, with templates, we use a single `StringTemplate` class. In both cases, the objects hold the data necessary to generate the associated output constructs. The only (minor) advantage to using specialized generator classes is that we can name the data fields.

Each generator class has a method, usually `toString()`, that renders the object to text in the right format. The typical generator class look something like this:

```
class «OutputConstruct» {
    «field1»
    «field2»
    ...
    «fieldN»
    public «OutputConstruct»(«field args») { «set fields» }
    public String toString() { «compute string from fields» }
}
```

Let's think about generating HTML. We could build a library with classes such as `Document`, `Head`, `Title`, `Table`, `ListItem`, and so on. `Title` instances would hold a title string, for example. Its `toString()` method would emit `<title>«a-title»</title>`.

It's reasonable to make generator classes handle slight differences in the output language, such as different versions of HTML. But, it's best not to think of these generator classes as generating radically different output according to some parameter. These classes are part of the output model. Their job is simply to emit text in the right format, not perform a translation.

Implementation

To demonstrate generator classes, let's build `Table` and `Column` classes to represent SQL table creation statements. An SQL table needs to know the table name and the list of columns:

```
public class Table {
    String name; // SQL table name
    List<Column> columns = new ArrayList<Column>();
    public String toString() {
        StringBuffer buf = new StringBuffer();
        buf.append("CREATE TABLE "+ name+" (\n");
```

```

        int i = 0;
        for (Column c : columns) {
            if ( i>0 ) buf.append(",\n");
            buf.append("    "); // indent a bit
            buf.append(c);
            i++;
        }
        buf.append(");\n");
        return buf.toString();
    }
}

```

For each column, we need to know its name, its type, and the list of attributes like NOT NULL and UNIQUE:

```

public class Column {
    String name;                // SQL column name
    String type;                // SQL column type
    List<String> attrs;          // SQL column attributes
    public String toString() {
        StringBuffer attrBuf = new StringBuffer();
        int i = 0;
        for (String a : attrs) {
            if ( i>0 ) attrBuf.append(", ");
            attrBuf.append(a);
            i++;
        }
        return name+" "+type+" "+attrBuf.toString();
    }
}

```

To use these classes, we create a Table and then Column instances for each column and fill in their fields. To generate text, we'd print the result of calling `toString()` on the Table instance.

Instead of creating strings in `toString()`, we could use templates. In that case, though, we might as well avoid the special generator classes and just use templates all by themselves.

Related Patterns

These generator classes are sort of like hard-coded templates.

Up Next

This chapter gives you the big picture on computer language translation. In the next chapter, we're going to drill down further into the

model-driven approach. We'll learn how to use templates to generate a number of DSLs and programming languages. The examples feed off of a variety of input models to create template-based output models.

Before we move on, let me highlight the two important lessons from this chapter. First, to properly translate an input phrase, we can't just cleverly replace input symbols with output symbols. We have to fully understand the input syntactically and semantically and then pick the proper output construct. Second, when using the model-driven approach, we need to decouple input model traversal order from the order dictated by the output language. This input-driven approach lets us compute information and translate phrases when it's convenient and efficient to do so.